🤖 Modele Machine Learning

Klasyfikacja, regresja i ewaluacja — jak poznać, że model naprawdę działa? 🎯

Python w Zastosowaniach Big Data | Wykład 8

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Szybkie przypomnienie z Wykładu 7 — Feature Engineering
2. Klasyfikacja vs regresja — dwa światy supervised learning
3. Przegląd algorytmów: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting
4. Parametry vs hiperparametry
5. Bias-variance tradeoff — overfitting i underfitting
6. Ewaluacja klasyfikacji: accuracy, precision, recall, F1, ROC-AUC, confusion matrix
7. Ewaluacja regresji: MAE, RMSE, R^2
8. Cross-validation — jak nie oszukać samego siebie
9. Najczęstsze pułapki i podsumowanie

> 🎯 **Po tym wykładzie:** Wytrenujesz kilka modeli klasyfikacyjnych, porównasz je uczciwie i wybierzesz najlepszy — nie na oko, tylko na podstawie metryk!

🔄 Szybkie przypomnienie z Wykładu 7

Na poprzednich zajęciach (Feature Engineering) nauczyliśmy się:

- Jak tworzyć nowe cechy z surowych danych (binning, interakcje, agregacje)
- Jak kodować zmienne kategoryczne (`OneHotEncoder` , `LabelEncoder`)
- Jak skalować cechy numeryczne (`StandardScaler` , `MinMaxScaler`)
- Jak budować `Pipeline` łączący preprocessing z modelem

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

> 💡 Dziś zakładamy, że dane są już **przygotowane** — skupiamy się na tym, co dzieje się **po** feature engineeringu: budowa modelu i ocena, czy jest dobry.

💡 Dlaczego to ważne?

Wyobraź sobie, że budujesz model, który mówi: "ten klient odejdzie z siłowni" z **99% accuracy**. Brzmi świetnie, prawda? 🎉

Ale co, jeśli tylko **1% klientów** faktycznie rezygnuje? Model, który **zawsze** odpowiada "nie odejdzie", też miałby 99% accuracy — i byłby **całkowicie bezużyteczny**! 😱

> 🎯 To dokładnie dlatego dzisiejszy wykład istnieje: **liczba nie kłamie, ale źle dobrana metryka potrafi oszukać nawet doświadczonego data scientistę.**

🏋️ Zbiór danych na dziś

Dziś (i na ćwiczeniach) pracujemy na syntetycznym zbiorze danych siłowni fitness 🏋️:

Kolumna	Opis
`wiek`	Wiek klienta (lata)
`plec`	Płeć klienta
`dlugosc_czlonkostwa_mies`	Ile miesięcy klient jest członkiem
`wizyty_miesiecznie`	Średnia liczba wizyt na miesiąc
`ocena_satysfakcji`	Ocena satysfakcji 1-10
`liczba_zamrozen`	Ile razy klient zamroził karnet
`zajecia_grupowe`	Czy korzysta z zajęć grupowych (bool)
`churn`	Czy klient zrezygnował (0/1) — **target**

> 🎯 Cel: przewidzieć `churn`, zanim klient faktycznie odejdzie — żeby siłownia mogła zareagować (rabat, telefon, nowa oferta) 📞

🧠 Klasyfikacja vs regresja — przypomnienie

	Klasyfikacja	Regresja
Co przewidujemy	Kategorię (klasę)	Liczbę ciągłą
Przykład	`churn` (0/1)	Przewidywana liczba wizyt/miesiąc
Typowe algorytmy	Logistic Regression, Decision Tree, Random Forest	Linear Regression, Decision Tree Regressor
Metryki	accuracy, precision, recall, F1, ROC-AUC	MAE, RMSE, R^2

> 💡 Wiele algorytmów (Decision Tree, Random Forest, Gradient Boosting) ma **obie wersje** — klasyfikacyjną i regresyjną! W scikit-learn to np. `RandomForestClassifier` vs `RandomForestRegressor`.

🔑 Algorytm #1: Logistic Regression

Mimo nazwy "Regression" — to algorytm **klasyfikacyjny**! Liczy prawdopodobieństwo przynależności do klasy, korzystając z funkcji sigmoidalnej.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Prawdopodobieństwo, nie tylko etykieta!
probs = model.predict_proba(X_test)  # kolumna 1 = P(churn=1)
```

Plusy: szybki, interpretowalny (współczynniki mówią o kierunku wpływu), dobry baseline.
Minusy: zakłada w miarę liniowe zależności między cechami a logitem prawdopodobieństwa.

🌳 Algorytm #2: Decision Tree

Model w formie **drzewa decyzyjnego** — seria pytań typu "czy `wizyty_miesiecznie` < 4?" prowadząca do liścia z predykcją.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)
```

Plusy: bardzo interpretowalny (można narysować!), obsługuje nieliniowe zależności, nie wymaga skalowania cech.
Minusy: bez ograniczeń (`max_depth`) **łatwo się przeucza** — zapamiętuje dane treningowe zamiast uczyć się wzorców.

> 🤔 **Pomyśl:** Drzewo bez `max_depth` może stworzyć osobny liść dla każdego klienta. Czy to dobrze uogólnia się na nowych klientów?

🌲 Algorytm #3: Random Forest

Las wielu drzew decyzyjnych, każde wytrenowane na losowej próbce danych i losowym podzbiorze cech — a na końcu **głosowanie** (klasyfikacja) lub **średnia** (regresja).

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,   # liczba drzew w lesie
    max_depth=10,
    random_state=42,
)
model.fit(X_train, y_train)
```

Plusy: znacznie mniej podatny na przeuczenie niż pojedyncze drzewo, dobra jakość "z pudełka", daje `feature_importances_`.
Minusy: mniej interpretowalny niż pojedyncze drzewo, wolniejszy trening.

> 💡 "Mądrość tłumu" — pojedyncze drzewo może się mylić, ale **200 drzew głosujących razem** rzadko myli się jednocześnie.

🚀 Algorytm #4: Gradient Boosting

Zamiast trenować drzewa **niezależnie** (jak Random Forest), Gradient Boosting trenuje je **sekwencyjnie** — każde kolejne drzewo poprawia błędy poprzednich.

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.1,   # 👈 jak mocno każde drzewo "poprawia" poprzednie
    max_depth=3,
    random_state=42,
)
model.fit(X_train, y_train)
```

Plusy: często najwyższa jakość predykcji (stąd popularność XGBoost/LightGBM w konkursach Kaggle).
Minusy: wolniejszy trening, więcej hiperparametrów do strojenia, łatwiej o przeuczenie przy zbyt dużym `n_estimators`.

🆚 Porównanie algorytmów — ściągawka

Algorytm	Szybkość treningu	Interpretowalność	Odporność na overfitting	Typowa jakość
Logistic Regression	⚡⚡⚡	★★★	★★★	Dobra (baseline)
Decision Tree	⚡⚡⚡	★★★	★	Zmienna
Random Forest	⚡⚡	★★	★★★	Bardzo dobra
Gradient Boosting	⚡	★	★★	Najlepsza (przy dobrym strojeniu)

> 🎯 **Zasada kciuka:** zacznij od Logistic Regression jako baseline, potem sprawdź Random Forest — a Gradient Boosting zostaw na koniec, kiedy chcesz wycisnąć ostatnie procenty jakości.

⚙️ Parametry vs hiperparametry

To rozróżnienie myli nawet doświadczonych studentów — warto je utrwalić raz na zawsze!

	Parametry	Hiperparametry
Kto je ustala?	Algorytm — automatycznie podczas `fit()`	Ty — **przed** treningiem
Przykład	Współczynniki w Logistic Regression	`max_depth`, `n_estimators`, `learning_rate`
Gdzie żyją w kodzie	`model.coef_`, `model.feature_importances_`	Argumenty konstruktora `RandomForestClassifier(...)`

```
model = RandomForestClassifier(n_estimators=200)  # 👈 hiperparametr — Ty decydujesz
model.fit(X_train, y_train)                       # 👈 parametry — model się ich uczy
print(model.feature_importances_)                 # 👈 wyuczony "parametr"
```

⚖️ Bias-Variance Tradeoff

Każdy model balansuje między dwoma rodzajami błędu:

- **Bias (obciążenie)** — model jest **zbyt prosty**, żeby uchwycić wzorzec → **underfitting**
- **Variance (wariancja)** — model jest **zbyt złożony**, uczy się szumu zamiast sygnału → **overfitting**

```
Prosty model  →  wysoki bias, niska wariancja  →  underfitting
Złożony model →  niski bias, wysoka wariancja  →  overfitting
```

> 💡 Pomyśl o tym jak o uczeniu się do egzaminu: **bias** to student, który uczy się tylko z jednej strony podręcznika (za mało). **Variance** to student, który zapamiętał na pamięć konkretne zadania z zeszłego roku, ale nie rozumie zasad (za dużo, źle).

📉 Underfitting vs Overfitting — jak to wygląda

Symptom	Underfitting	Overfitting
Accuracy na **train**	Niska	Bardzo wysoka (~100%)
Accuracy na **test**	Niska	Znacznie niższa niż train
Typowa przyczyna	Zbyt prosty model, za mało cech	Zbyt złożony model, za mało danych
Rozwiązanie	Bardziej złożony model, więcej cech	Regularyzacja, mniej głębokie drzewa, więcej danych

```
print(f"Train accuracy: {model.score(X_train, y_train):.2%}")
print(f"Test accuracy:  {model.score(X_test, y_test):.2%}")
# Duża różnica (train >> test) = 🚨 sygnał overfittingu!
```

🤔 Think about it

Pytanie: Twój `DecisionTreeClassifier` bez `max_depth` ma 100% accuracy na zbiorze treningowym i 68% na testowym. `LogisticRegression` ma 82% na obu. Który model wdrożysz produkcyjnie i dlaczego?

🎯 Confusion Matrix — fundament ewaluacji klasyfikacji

Zanim policzymy jakąkolwiek metrykę, musimy zrozumieć **macierz pomyłek**:

	Przewidziano: brak churn	**Przewidziano: churn**
Rzeczywiście: brak churn	True Negative (TN)	False Positive (FP)
Rzeczywiście: churn	False Negative (FN)	True Positive (TP)

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)
# [[TN, FP],
#  [FN, TP]]
```

> 🎯 **FP** = fałszywy alarm (siłownia dzwoni do klienta, który i tak by został — kosztowne, ale nie tragiczne). **FN** = przegapiony odchodzący klient (tracimy go bez interwencji — gorsze!).

📊 Accuracy — najprostsza, ale zdradliwa metryka

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
```

> ⚠️ **Pułapka:** przy **niezbalansowanych klasach** (np. 95% klientów zostaje, 5% odchodzi), model zawsze przewidujący "zostaje" ma 95% accuracy — i **zero wartości biznesowej**!

To dokładnie sytuacja z naszego przykładu z gymu — dlatego potrzebujemy więcej metryk 👇

🎯 Precision i Recall

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

```
from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_test, y_pred)  # spośród przewidzianych "churn" – ile trafnych?
recall = recall_score(y_test, y_pred)        # spośród faktycznych "churn" – ile wykryliśmy?
```

Metryka	Pytanie, na które odpowiada	Kiedy priorytet
Precision	Ile alarmów było trafnych?	Gdy FP jest kosztowny (spam)
Recall	Ile prawdziwych przypadków wykryliśmy?	Gdy FN jest kosztowny (choroba, churn)

> 💡 W przypadku churnu **recall zwykle jest ważniejszy** — lepiej niepotrzebnie zadzwonić do 10 klientów niż przegapić 1 realnie odchodzącego.

⚖️ F1-score — kompromis między precision a recall

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
from sklearn.metrics import f1_score, classification_report

f1 = f1_score(y_test, y_pred)
print(classification_report(y_test, y_pred))  # 👈 wszystko w jednym miejscu!
```

Dlaczego średnia harmoniczna, a nie zwykła? Bo kara za skrajności — jeśli precision = 100%, ale recall = 1%, zwykła średnia dałaby ~50%, a F1 słusznie da **bardzo niski wynik**.

📈 ROC Curve i AUC

ROC curve pokazuje kompromis między **True Positive Rate** (recall) a **False Positive Rate** przy różnych progach decyzyjnych.

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

probs = model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, probs)
auc = roc_auc_score(y_test, probs)

plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")
plt.plot([0, 1], [0, 1], "--", label="Losowy model")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
```

> 🎯 **AUC = 0.5** → model losowy (rzut monetą). **AUC = 1.0** → model idealny. W praktyce dobre modele mieszczą się w **0.75-0.95**.

🎚️ Threshold tuning — próg decyzyjny to też Twój wybór

Domyślnie `predict()` używa progu **0.5**, ale to tylko konwencja!

```
probs = model.predict_proba(X_test)[:, 1]

# Obniżamy próg — łapiemy więcej churnu kosztem więcej fałszywych alarmów
y_pred_niski_prog = (probs >= 0.3).astype(int)
```

Próg	Recall	Precision	Kiedy stosować
Niski (0.2-0.3)	Wysoki	Niski	Gdy przegapienie churnu jest bardzo kosztowne
Wysoki (0.7-0.8)	Niski	Wysoki	Gdy fałszywy alarm jest kosztowny (np. drogi rabat)

> 💡 Ta sama macierz cech (i model!) może dawać różne metryki w zależności od **jednej liczby** — progu decyzyjnego.

📏 Ewaluacja regresji — przypomnienie z Wykładu 4

Gdybyśmy przewidywali liczbę (np. `wizyty_miesiecznie`), użylibyśmy innych metryk:

$$\text{MAE} = \frac{1}{n}\sum|y_i - \hat{y}_i| \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$$

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)
```

Metryka	Interpretacja
MAE	Średni błąd bezwzględny — łatwy do wytłumaczenia biznesowi
RMSE	Jak MAE, ale mocniej karze duże pomyłki (podniesienie do kwadratu)
R^2	Jaki % wariancji target wyjaśnia model (0 = tak słabo jak średnia, 1 = idealnie)

🔀 Cross-Validation — dlaczego jeden podział to za mało

Pojedynczy `train_test_split()` może "trafić" na łatwy albo trudny podział przez przypadek. **K-fold cross-validation** dzieli dane na K części i trenuje/testuje K razy.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5, scoring="f1")
print(f"F1 per fold: {scores.round(3)}")
print(f"Średnia: {scores.mean():.3f} ± {scores.std():.3f}")
```

```
Fold 1: [train train train train | TEST]
Fold 2: [train train train | TEST | train]
Fold 3: [train train | TEST | train train]
...
```

> 🎯 Dostajesz **5 niezależnych ocen** modelu zamiast jednej — dużo bardziej wiarygodny obraz jakości!

🔍 GridSearchCV — automatyczne strojenie hiperparametrów

Zamiast ręcznie próbować kombinacji `max_depth=5`, potem `max_depth=10`... niech komputer zrobi to za nas!

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [5, 10, 15],
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="f1",
)
grid_search.fit(X_train, y_train)

print(f"Najlepsze parametry: {grid_search.best_params_}")
print(f"Najlepszy F1: {grid_search.best_score_:.3f}")
```

> ⚠️ `GridSearchCV` sprawdza **wszystkie** kombinacje — przy dużej siatce może być wolny. Alternatywa: `RandomizedSearchCV`.

🌟 Feature Importance — co model uznał za ważne?

Modele oparte na drzewach dają za darmo informację o tym, **które cechy** miały największy wpływ na predykcję.

```
import pandas as pd

importances = pd.Series(
    model.feature_importances_, index=X_train.columns
).sort_values(ascending=False)

importances.plot(kind="barh")
```

> 💡 Dla naszego przykładu z siłownią spodziewalibyśmy się, że `wizyty_miesiecznie` i `ocena_satysfakcji` będą górować nad `plec` — jeśli tak nie jest, warto zbadać dane głębiej!

⚠️ Częste pułapki #1: Data Leakage

Data leakage = informacja z przyszłości (albo z targetu) "przecieka" do cech treningowych.

```
# ❌ ŹLE: skalujemy PRZED podziałem train/test
scaler.fit(X)  # 🚨 scaler "widział" dane testowe!
X_scaled = scaler.transform(X)
X_train, X_test = train_test_split(X_scaled, ...)

# ✅ DOBRZE: dzielimy najpierw, potem dopasowujemy scaler TYLKO na train
X_train, X_test = train_test_split(X, ...)
scaler.fit(X_train)                # 👈 tylko train!
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)   # transform, nie fit!
```

> 🎯 `Pipeline` z Wykładu 7 rozwiązuje ten problem automatycznie — `cross_val_score(pipeline, ...)` dopasowuje preprocessing osobno w każdym foldzie!

⚠️ Częste pułapki #2: Niezbalansowane klasy

Gdy jedna klasa dominuje (np. 95% "brak churn"), model może "oszukiwać" przez ignorowanie klasy mniejszościowej.

Strategia	Jak działa
`class_weight="balanced"`	Model karze mocniej za błędy na klasie mniejszościowej
Oversampling	Powielanie (lub syntetyczne generowanie) przykładów klasy mniejszościowej
Undersampling	Usuwanie części przykładów klasy większościowej
Zmiana metryki	Patrz na F1/recall, nie na accuracy

```
model = RandomForestClassifier(class_weight="balanced", random_state=42)
```

⚠️ Częste pułapki #3: Porównywanie modeli na różnych podziałach

```
# ❌ ŹLE: każdy model dostaje INNY losowy podział train/test
model_a.fit(*train_test_split(X, y))
model_b.fit(*train_test_split(X, y))  # 🚨 inny podział = nieuczciwe porównanie!

# ✅ DOBRZE: ten sam random_state, ten sam podział dla wszystkich modeli
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
model_a.fit(X_train, y_train)
model_b.fit(X_train, y_train)
```

> 💡 To samo dotyczy `cross_val_score` — zawsze ustawiaj `random_state`, żeby wyniki były powtarzalne i porównywalne!

🧠 Think about it

Pytanie: Masz dwa modele — jeden ma accuracy 90% i recall 40%, drugi ma accuracy 85% i recall 75%. Siłownia chce **zatrzymać jak najwięcej** odchodzących klientów. Który model wybierzesz?

💻 Demo end-to-end — cały pipeline w jednym miejscu

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(n_estimators=200, random_state=42)),
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
probs = pipeline.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, probs):.3f}")
```

> 🎯 Zauważ `stratify=y` — zachowuje proporcję klas w train i test, kluczowe przy niezbalansowanych danych!

Kluczowe wnioski

1. ✅ **Klasyfikacja** przewiduje kategorie, **regresja** przewiduje liczby — ale wiele algorytmów ma obie wersje
2. ✅ **Accuracy to za mało** — przy niezbalansowanych klasach zawsze patrz na precision, recall, F1, ROC-AUC
3. ✅ **Bias-variance tradeoff** — zbyt prosty model = underfitting, zbyt złożony = overfitting
4. ✅ **Cross-validation** daje wiarygodniejszy obraz jakości niż pojedynczy `train_test_split`
5. ✅ **Hiperparametry** stroisz Ty (`GridSearchCV` pomaga), **parametry** uczy się model

⚠️ Ściągawka: unikaj tych błędów

Pułapka	Rozwiązanie
Skalowanie przed podziałem train/test	Zawsze najpierw `train_test_split`, potem `fit` scalera na train
Optymalizacja tylko pod accuracy	Sprawdź F1/recall przy niezbalansowanych klasach
Brak `random_state`	Ustawiaj wszędzie — powtarzalność to podstawa nauki
Ignorowanie feature importance	Zawsze sprawdź, czy model "uczy się" sensownych rzeczy

🔮 Co dalej?

Na dzisiejszych **ćwiczeniach** zbudujesz model przewidujący rezygnacje klientów siłowni 🏋️ — od baseline `LogisticRegression`, przez `RandomForestClassifier`, aż po strojenie hiperparametrów z `GridSearchCV`.

Następny wykład (Wykład 9, 01.12.2026): Spark MLlib — Machine Learning na dużą skalę! Ten sam pipeline (features → model → ewaluacja), ale na danych, które nie mieszczą się na jednym komputerze 🚀

❓ Pytania?

Dziękuję za uwagę! 🎉

Zapraszam na ćwiczenia — czas budować i oceniać modele! 💻🏋️